UNIVERSIDADE VILA VELHA

Curso de Ciência da Computação

Campus Boa Vista

Atividades Avaliativas — POO

Documentação Técnica dos Projetos

Atividade 01 — Espírito Guerreiro (Sistema Gamer)

Atividade 02 — POO bet (Sistema de Apostas)

Data de Entrega: 23 de Abril de 2026

Arthur D'Avila, Arthur Adnet Felix, Cauã Passos Gomes, David Evora

Sumário

1. Introdução
2. Atividade 01 — Espírito Guerreiro
   2.1. Descrição do Projeto
   2.2. Estrutura de Classes
   2.3. Requisitos Implementados
   2.4. Fluxo do Jogo
3. Atividade 02 — POO bet
   3.1. Descrição do Projeto
   3.2. Estrutura de Classes
   3.3. Requisitos Implementados
4. Uso de Inteligência Artificial (AI Assessment Scale)
   4.1. Ferramentas e Prompts Utilizados
   4.2. Critérios de Revisão Humana
   4.3. Nível Empregado
   4.4. Evidências Comparativas
   4.5. Reflexões sobre Validade e Limitações
5. Considerações Finais

1. Introdução

Este documento apresenta a documentação técnica referente às duas atividades avaliativas da disciplina de Programação Orientada a Objetos (POO) do Curso de Ciência da Computação da Universidade Vila Velha — Campus Boa Vista.

O objetivo geral das atividades é implementar soluções algorítmicas utilizando o paradigma orientado a objetos em Java, abrangendo os seguintes conceitos:

- Classes e Objetos
- Conjunto de Atributos e Comportamentos
- Associação e Navegabilidade entre classes
- Coleções (List, ArrayList)
- Lógica Computacional
- Janelas de Interface Gráfica via classe InOut (JOptionPane)

Ambos os projetos foram desenvolvidos em Java, utilizando a classe InOut para todas as entradas e exibições ao usuário via janelas de diálogo.

---

2. Atividade 01 — Espírito Guerreiro

2.1. Descrição do Projeto

O projeto Espírito Guerreiro é um sistema gamer baseado em texto com interface gráfica via JOptionPane. O jogo é conduzido por um personagem chamado Oráculo, que guia o Guerreiro por três níveis de desafios. O sistema simula uma aventura com vidas, penalidades por erro, sistema de misericórdia e relatório final.

O projeto foi desenvolvido utilizando IntelliJ IDEA com Java 24 e utiliza exclusivamente a classe InOut.java para toda interação com o usuário.

2.2. Estrutura de Classes

O diagrama de classes do sistema é composto pelas seguintes entidades:

Classe	Responsabilidade
Oraculo	Classe central do jogo. Controla os níveis, vidas, prólogos e lógica do jogo.
Guerreiro	Representa o jogador. Armazena nome, quantidade de vidas (qtdVidas) e a bolsa.
Bolsa	Associada ao Guerreiro (1 para 1). Gerencia itens do guerreiro (equipar/desequipar).
Item	Representa um item do jogo, com id, tipo e estado de equipamento.

2.3. Requisitos Implementados

Oráculo — Atributos e Métodos

Membro	Descrição
- nome : String	Nome do Oráculo, definido via definirNome()
- warrior : Guerreiro	Referência ao Guerreiro associado ao Oráculo
void definirNome(String)	Seta o nome do Oráculo
int sortearVidas()	Sorteia vidas entre [9, 12] usando Random
String prologoIntroducao()	Exibe prólogo com nomes do Oráculo e Guerreiro + quantidade de vidas
boolean loadLevel1()	Jogo de adivinhação de número [1–100] com dicas MENOR/MAIOR. Cada erro = -1 vida
boolean loadLevel2()	Jogo de charadas (mínimo 3). Cada erro = -1 vida
boolean loadLevel3()	Minigame proposto pelo grupo (alinhado com professor)
String vidaExtra()	Lê pedido de misericórdia via I/O
boolean decidirVidaExtra(String)	Concede +1 vida se o pedido tiver mais de 5 palavras
String prologoVencedor()	Exibe mensagem de vitória
String prologoPerdedor()	Exibe mensagem de derrota
String RelatorioFimGame()	Exibe relatório final: vidas perdidas, palpites do Level1, resposta certa do Level2, etc.

Guerreiro — Atributos e Métodos

Membro	Descrição
- nome : String	Nome do guerreiro
- qtdVidas : int	Quantidade de vidas, sorteada pelo Oráculo
- myBolsa : Bolsa	Referência à Bolsa do guerreiro (associação 1 para 1)
String vidaExtra()	Lê pedido de misericórdia do jogador

Bolsa e Item

Membro	Descrição
Bolsa.item : Item	Referência ao item (associação 1 para *) com ArrayList
Bolsa.equipar()	Equipa um item da bolsa
Bolsa.desequipar()	Desequipa um item da bolsa
Item.idItem : int	Identificador único do item
Item.tipoItem : String	Tipo/nome do item
Item.equipado : boolean	Estado do item
Item.imprimirDados()	Exibe os dados do item via InOut

2.4. Fluxo do Jogo

Na classe Principal (main), é instanciado um objeto Oráculo Master. O jogo executa no mínimo duas partidas completas, seguindo o fluxo:

- Prólogo de introdução com nomes e vidas sorteadas
- Level 1: Adivinhação de número com dicas (MENOR/MAIOR)
- Level 2: Jogo de charadas com pelo menos 3 perguntas
- Level 3: Minigame customizado pelo grupo
- Ao zerar as vidas: sistema oferece pedido de misericórdia
- Oráculo decide se concede vida extra (mais de 5 palavras no pedido)
- Exibição de prólogo de vencedor ou perdedor
- Relatório final com estatísticas da partida

Desafio implementado: execução de arquivo .WAV nos prólogos usando javax.sound.sampled.AudioSystem para sons de vitória e derrota.

---

3. Atividade 02 — POO bet

3.1. Descrição do Projeto

O POO bet é um sistema de apostas esportivas com interface gráfica via JOptionPane. O jogador se cadastra, consulta jogos disponíveis, realiza apostas com valor controlado por saldo de crédito, vê resultados aleatórios e gerencia suas apostas (CRUD). O sistema implementa associação entre Jogador, Credito, Aposta e Jogo.

3.2. Estrutura de Classes

Classe	Responsabilidade
Jogador	Entidade principal. Contém cadastro, menu principal e gerenciamento de apostas.
Credito	Associado 1:1 ao Jogador. Controla saldo, validade, moeda e bloqueio.
Aposta	Associada * ao Jogador. Representa uma aposta feita em um ou mais Jogos.
Jogo	Lista estática de jogos disponíveis para aposta, com odd, apostaMáx e prêmioMáx.

3.3. Requisitos Implementados

Jogador

Membro	Descrição
- idJogador : int	ID auto-incrementado via contadorId estático
- nome, sobrenome : String	Dados pessoais do jogador
- apelido, cpf : String	Identificadores do jogador

- nacionalidade : String	Nacionalidade do jogador
- dataNascimento : String	Data de nascimento
- credito : Credito	Referência ao crédito do jogador (1:1)
- minhasApostas : List<Aposta>	Coleção de apostas do jogador
void cadastrar()	Lê todos os dados via InOut e exibe saldo inicial de 500
void menu()	Menu principal com 7 opções via loop while

Funcionalidades do Menu

Opção	Funcionalidade
1 — Ver apostas disponíveis	Chama Jogo.imprimirJogos() — lista todos os jogos com ID, descrição, apostaMax e odd
2 — Apostar	Lê ID do jogo e valor. Valida saldo, bloqueio e limite da aposta. Deduz valor do saldo
3 — Ver minhas apostas	Itera sobre minhasApostas e chama verApostas() para cada uma
4 — Resultado	Processa resultado aleatório de cada aposta, atualiza saldo e limpa a lista
5 — Excluir apostas	Remove aposta pelo ID informado
6 — Usuário	Exibe dados pessoais e saldo atual
7 — Sair	Encerra o loop do menu

Credito

Membro	Descrição
- saldo : double	Saldo inicial de 500. Decrementado ao apostar, incrementado ao ganhar
- validade : String	Validade do crédito
- moeda : String	Moeda utilizada
- bloqueado : boolean	Se true, nenhuma aposta pode ser realizada

Aposta

Membro	Descrição
- idAposta : int	Auto-incrementado
- valorAposta : double	Valor apostado
- repetir : int	Quantidade de repetições
- jogosApostados : List<Jogo>	Lista de jogos incluídos nesta aposta
void apostarJogo(int idJogo)	Busca o jogo na lista estática e adiciona à aposta

void verApostas()	Exibe jogos apostados e valor via InOut
void excluirAposta(int idJogo)	Remove jogo da aposta pelo id
double resultadoDaAposta()	Retorna o multiplicador de odds se ganhou, 0 se perdeu (Random)

Jogo

Membro	Descrição
jogos : List<Jogo>	Lista estática com 5 jogos pré-cadastrados
idJogo, descritivo	Identificador e descrição do evento esportivo
apostaMax, premioMax	Limites de aposta e prêmio
odd : double	Multiplicador de ganho
static imprimirJogos()	Exibe todos os jogos via InOut
static getJogoById(int)	Busca jogo por ID com stream().filter()
boolean resultado()	Retorna resultado aleatório via new Random().nextBoolean()

Jogos disponíveis no sistema:

- ID 1 — Flamengo vai ganhar do Palmeiras? (odd: 5.0, max: R$500)
- ID 2 — Lakers vai ganhar do Warriors? (odd: 6.0, max: R$300)
- ID 3 — FURIA vai ganhar da NAVI no CS2? (odd: 7.5, max: R$200)
- ID 4 — Max Verstappen vai ganhar o GP do Brasil? (odd: 7.5, max: R$400)
- ID 5 — Real Madrid vai ganhar do Manchester City? (odd: 5.8, max: R$600)

---

4. Uso de Inteligência Artificial — AI Assessment Scale

4.1. Ferramentas e Prompts Utilizados

Ferramenta utilizada: Claude (Anthropic) — interface web claude.ai

Exemplos de prompts utilizados durante o desenvolvimento:

- "Como usar Random para sortear um número entre 9 e 12 em Java?" — consulta de sintaxe
- "Como usar stream().filter() para buscar um objeto em uma List por ID?" — exploração de Collections
- "Como estruturar a relação entre Aposta e Jogo usando ArrayList em Java?" — planejamento de associação
- "Quais são as diferenças entre List.of() e new ArrayList() em Java?" — descoberta conceitual
- "Como reproduzir um arquivo .WAV em Java com javax.sound.sampled?" — pesquisa para o desafio

A IA foi utilizada exclusivamente como ferramenta de descoberta e planejamento, conforme previsto no nível IA-2 da escala.

4.2. Critérios de Revisão Humana

Todos os trechos de código gerados ou sugeridos pela IA passaram pelos seguintes critérios de revisão humana antes de serem incorporados ao projeto:

- Compreensão completa da lógica antes de utilizar qualquer trecho
- Verificação de que o código compila e executa corretamente no IntelliJ/NetBeans
- Adaptação ao padrão de nomeação e estrutura definidos pelo professor
- Teste manual de cada funcionalidade via janelas JOptionPane
- Conferência de que os requisitos do diagrama de classes foram respeitados

Declaração explícita: 'A IA auxiliou na geração inicial de código com revisão humana completa.'

4.3. Nível Empregado

Nível utilizado: IA-2 (Automação Guiada).

A IA foi utilizada para automatizar partes da tarefa (ex.: sugerir estruturas de código, métodos e sintaxe Java), mas com supervisão humana contínua. Todas as decisões de design de classe, lógica de negócio e integração entre os objetos foram tomadas pelo grupo.

4.4. Evidências Comparativas

Exemplo: resultadoDaAposta() — SEM IA (versão inicial)

```
public double resultadoDaAposta() {
    // Versao manual: verificava apenas o primeiro jogo
    Jogo j = jogosApostados.get(0);
    if (j.resultado()) {
        return valorAposta * j.getOdd();
    }
    return 0;
}
```

Exemplo: resultadoDaAposta() — COM IA (versão refatorada)

```
public double resultadoDaAposta() {
    for (Jogo j : jogosApostados) {
        boolean ganhou = j.resultado();
        if (ganhou) {
            return j.getOdd();  // retorna o multiplicador
        }
    }
    return 0;
}
```

Ajuste realizado: a versão com IA sugeria iterar sobre todos os jogos apostados, o que foi adotado. No entanto, o grupo identificou que retornar apenas o multiplicador (odd) em vez do valor final é mais correto para a arquitetura atual, onde o cálculo do prêmio ocorre em Jogador.menu().

Tarefa realizada sem IA (IA-0)

Declaração: A modelagem inicial das classes Guerreiro, Bolsa e Item (definição de atributos, associações e relações de navegabilidade conforme o diagrama UML) foi realizada manualmente, sem uso de IA. A leitura e interpretação do enunciado e do diagrama de classes também foi realizada manualmente pelo grupo.

4.5. Reflexões sobre Validade e Limitações

- Validade: A IA foi útil para esclarecer dúvidas de sintaxe Java e sugerir uso de Collections. As sugestões foram sempre revisadas e adaptadas ao contexto do projeto.
- Limitação: A IA não conhece os requisitos específicos do professor, especialmente o Level 3 do jogo (minigame customizado), que precisou ser discutido diretamente com o professor e implementado manualmente.
- Limitação: Sugestões da IA para estrutura de classes às vezes diferiam do diagrama UML do enunciado. O diagrama sempre prevaleceu.
- Reflexão: O uso de IA como ferramenta de descoberta acelerou a compreensão de APIs como javax.sound.sampled e java.util.stream, mas a implementação final demandou adaptação crítica do grupo.

---

5. Considerações Finais

As duas atividades foram desenvolvidas com sucesso, implementando os conceitos de Programação Orientada a Objetos exigidos: classes e objetos, associações, coleções, encapsulamento, lógica computacional e interface gráfica via JOptionPane.

O projeto Espírito Guerreiro demonstra o uso de múltiplas classes interligadas em um contexto de jogo, com lógica de níveis, sistema de vidas e relatório de desempenho. O projeto POO bet demonstra CRUD de apostas, controle de saldo, uso de List<T> e busca com Streams.

O uso de IA foi mantido dentro do nível IA-2 conforme exigido, com toda a responsabilidade intelectual e domínio do código mantidos pelo grupo. A documentação reflete com transparência onde e como a IA foi utilizada, com evidências comparativas e reflexão crítica sobre os outputs gerados.

Vila Velha, ES — Abril de 2026